Proof Rules for Termination of Loops: A Comparative Analysis

KRZYSZTOF R. APT, Centrum voor Wiskunde en Informatica, The Netherlands, MIMUW, Poland
FRANK S. DE BOER, Centrum voor Wiskunde en Informatica, The Netherlands
ERNST-RÜDIGER OLDEROG, Carl von Ossietzky Universität Oldenburg, Germany

In the context of Hoare's logic various variants of proof rules for proving termination of **while** programs were proposed. We provide here a comparative analysis of these rules and show their proof-theoretic equivalence. By introducing the distinction between admissible and derivable proof rules, we clarify the crucial role auxiliary rules play in such an analysis. By discussing representations of proofs in the form of proof outlines, we reveal differences between these equivalent proof rules when used in practice. We also address applications in the context of the paradigm of design by contract. Finally, we show how to extend the equivalence result to a larger class of programs that access a library of procedures.

CCS Concepts: • **Theory of computation → Hoare logic**.

Additional Key Words and Phrases: Program verification, Termination, Analysis of proof rules, Proof outlines

ACM Reference Format:
Krzysztof R. Apt, Frank S. de Boer, and Ernst-Rüdiger Olderog. 2026. Proof Rules for Termination of Loops: A Comparative Analysis. *Form. Asp. Comput.* 0, 0, Article 1 (January 2026), 28 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

In his seminal paper [17], Hoare introduced an axiomatic method of reasoning about correctness of **while** programs, now called Hoare's logic. It is based on correctness formulas $\{p\}\ S\ \{q\}$, where S is a program and p and q are assertions (logical formulas), with the interpretation

> "If the assertion p is true before initiation of a program S, then the assertion q will be true on its completion."

In this context p is referred to as a *precondition* and q as a *postcondition* of S.

However, in contrast to Floyd's earlier paper [13] that dealt with correctness of flowchart programs, program termination was not addressed. To stress this difference one distinguishes now between *partial correctness* that only focuses on the delivery of correct results, and *total correctness*, that in addition stipulates that the program terminates. So the original proposal of Hoare dealt with partial correctness.

All approaches to proving program termination within Hoare's logic formalize Floyd's [13] observation that

> "Proofs of termination are dealt with by showing that each step of a program decreases some entity which cannot decrease indefinitely."

Authors' Contact Information: Krzysztof R. Apt, Centrum voor Wiskunde en Informatica, Amsterdam, The Netherlands, and MIMUW, Warsaw, Poland, apt@cwi.nl; Frank S. de Boer, Centrum voor Wiskunde en Informatica, Amsterdam, The Netherlands, F.S.de.Boer@cwi.nl; Ernst-Rüdiger Olderog, Carl von Ossietzky Universität Oldenburg, Oldenburg, Germany, olderog@informatik.uni-oldenburg.de.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
Manuscript submitted to ACM

The first extension of Hoare's logic to deal with total correctness is due to [20]. Since then substantially simpler proof rules were proposed. In these proof rules variables that range over natural numbers are used. An appropriate relative completeness result, see for example [4], shows that variables ranging over more general well-founded orderings are not needed.

Termination continues to be a relevant and vibrant topic in program analysis, see for example [9] and the Annual International Termination Competition[1]. The latter comprises various competition categories, for instance proving termination of C programs, Java bytecode programs, logic programs, functional programs, and term rewriting systems. Here we focus on the shape of termination proofs in the context of Hoare's logic.

In this paper, we investigate three natural proof rules and their variants from the proof-theoretic point of view[2]. More specifically, we study **while** programs with Hoare's original proof system for program correctness, in which the well-known proof rule for partial correctness of the **while** loops, which we call the LOOP I rule, is replaced by a suitable proof rule for establishing total correctness. We analyze three versions of such a proof rule.

- The LOOP II rule involves a single, monolithic proof which establishes an invariant, termination of the body of the **while** loop, and which ensures that the body itself cannot be executed infinitely often by a so-called bound function.
- The LOOP III rule achieves a separation of the reasoning about the invariant and the bound function.
- The LOOP IV rule not only achieves a separation of the reasoning about the invariant and the bound function, it additionally involves a separate proof of the termination the body itself. As such it is particularly well-suited when dealing with nested loops because it modularizes the correctness proof of the outer loop from the ones of the inner loops. It is a *hybrid* proof rule with premises referring to proof systems for both partial and total correctness.

Depending on the choice of the loop rule, we obtain proof systems that we refer to by II, III, and IV, respectively. We show that these three proof systems are equivalent in the sense that every proof of a correctness formula $\{p\}\ S\ \{q\}$ carried out in one of these systems can be effectively transformed into a correctness proof in any other of these proof systems. This result is obtained by a detailed proof-theoretic analysis of the four loop rules in the context of these proof systems. In this analysis we make use of auxiliary proof rules which we show to be admissible in the proof systems.

Even though Hoare's logic has been extensively studied (see for example our survey [6]), little work has been done on the analysis of proofs in Hoare's logic. We are familiar with only three references, [2], [10], and [25], in which transformations of proofs in a Hoare logic are discussed.

While these proof rules are equivalent, their use and representation in the form of proof outlines, which are programs annotated by assertions, differs. We illustrate this by analyzing a termination proof of a program with nested loops. We also address applications in the context of assertions used as annotations in the design by contract paradigm. Finally, we show how to extend the equivalence result to a larger class of programs that access a library of procedures.

Our paper is organized as follows. In Section 2, we extend in three ways the classical proof rule for partial correctness of loops to deal with termination. We end by stating the main result of the paper: Equivalence Theorem 2.1. In Section 3, we discuss we origins and merits of the loop rules for termination and point to variants that will be analyzed in Appendix C. Section 4, is devoted to proving the Equivalence Theorem 2.1. We use the notions of admissibility and derivability of proof rules to establish appropriate lemmas about specific auxiliary rules. These lemmas provide the

[1] `https://termination-portal.org/wiki/Termination_Competition`
[2] This paper is a modified and extended version of [5].

blueprint for our equivalence proof. The auxiliary rules needed for our analysis are then introduced. The overall structure of the equivalence proof is outlined by referring to the central lemmas of derivability and admissibilty, which are proven in Appendix B. We close by discussing restrictions imposed on the bound functions in the loop rules for termination. In Section 5, we discuss the representation of proofs in the form of proof outlines and we reveal differences between the equivalent proof rules when used in practice. The termination proof of an example program serves to illustrate this. In Section 6, we point to applications of proof outlines in the context of the paradigm of design by contract, in particular the Java Modeling Language (JML). In Section 7, we show how to extend the equivalence result to a larger class of programs that access a library of procedures. In Section 8, we conclude our paper.

Three appendices complete the paper. Appendix A recalls the proof system for partial correctness of **while** programs. Appendix B gives all details of the proof of the Equivalence Theorem 2.1. Appendix C shows the equivalence of variants of the loop rules alluded to in Section 3.

2 LOOP rules

We shall be concerned here with the language, the formulas of which are either first-order formulas, called ***assertions***, or ***correctness formulas***, which are constructs of the form $\{p\}\ S\ \{q\}$, where p and q are assertions and S is a **while** program.

We assume two disjoint sets of variables: a set *lvar* of ***logical*** variables and a set *pvar* of ***program*** variables. Programs refer only to program variables, while assertions may refer to both types of variables. Logical variables are the only variables that can be quantified over. They are used in assertions as ***freeze*** variables. To keep things simple we assume that all program and logical variables are simple variables of type **integer**.

Below we denote by $free(p)$ the set of free variables of the assertion p and by $var(t)$, $var(B)$, and $var(S)$ the set of variables that appear in the expression t, the Boolean expression B, and the program S, respectively. We denote by $p[x := t]$ the result of substituting all free occurrences of the variable x in p by the expression t.

We shall consider in total four proof systems concerned with the correctness formulas. They only differ in the used LOOP rule.

Proof system I denotes the customary proof system allowing us to prove partial correctness of **while** programs. Its axioms and proof rules, due to [17], are listed in Appendix A. Its LOOP rule has the following form:

RULE LOOP I

$$\frac{\{p \wedge B\}\ S\ \{p\}}{\{p\}\ \textbf{while}\ B\ \textbf{do}\ S\ \textbf{od}\ \{p \wedge \neg B\}}$$

In the proof system II, this rule is replaced by

RULE LOOP II

$$\frac{\begin{array}{l}\{p \wedge B \wedge t = z\}\ S\ \{p \wedge t < z\},\\ p \rightarrow t \geq 0\end{array}}{\{p\}\ \textbf{while}\ B\ \textbf{do}\ S\ \textbf{od}\ \{p \wedge \neg B\}}$$

where t is an integer expression such that $var(t) \subseteq pvar$ and z is a *logical* integer variable that does not appear in p.

In the proof system III the LOOP I rule is replaced by

RULE LOOP III

$$\frac{\begin{array}{l}\{p \wedge B\}\ S\ \{p\},\\ \{p \wedge B \wedge t = z\}\ S\ \{t < z\},\\ p \rightarrow t \geq 0\end{array}}{\{p\}\ \textbf{while}\ B\ \textbf{do}\ S\ \textbf{od}\ \{p \wedge \neg B\}}$$

where t and z are as above.

In the context of the LOOP rules discussed here, the assertion p is called the ***loop invariant*** and the expression t is called the ***bound function***. It provides an estimate how many iterations the loop will still perform before termination.

We shall consider next the following hybrid rule that combines provability in two proof systems.

RULE LOOP IV

$$\frac{\begin{array}{l}\vdash_{\mathrm{I}} \{p \wedge B\}\ S\ \{p\},\\ \vdash_{\mathrm{I}} \{p \wedge B \wedge t = z\}\ S\ \{t < z\},\\ \{p \wedge B\}\ S\ \{\textbf{true}\},\\ p \rightarrow t \geq 0\end{array}}{\{p\}\ \textbf{while}\ B\ \textbf{do}\ S\ \textbf{od}\ \{p \wedge \neg B\}}$$

where t and z are as above.

Proof system IV is obtained from proof system I by replacing the LOOP I rule by the LOOP IV rule. The use of two forms of provability in the premises of this rule can be circumvented by the following modification of the notation. Denote the correctness formulas in the sense of partial correctness by $\{p\}\ S\ \{q\}$ and in the sense of total correctness by $[p]\ S\ [q]$. Then combine the proof system I with the proof system in which the axioms and proof rules of I except the LOOP I rule are rewritten using the $[p]\ S\ [q]$ syntax. Finally, add to this proof system the LOOP IV rewritten as follows:

$$\frac{\begin{array}{l}\{p \wedge B\}\ S\ \{p\},\\ \{p \wedge B \wedge t = z\}\ S\ \{t < z\},\\ [p \wedge B]\ S\ [\textbf{true}],\\ p \rightarrow t \geq 0\end{array}}{[p]\ \textbf{while}\ B\ \textbf{do}\ S\ \textbf{od}\ [p \wedge \neg B]}$$

where t and z are as above.

In what follows we use the original formulation of this rule, as it will not lead to any ambiguities.

The main aim of this paper is to prove the following result.

Theorem 2.1 (Equivalence). *The proof systems II-IV are equivalent.*

Here, of course, equivalence of the proof systems means that the same formulas can be proved in each of them.

3 Discussion

Before we proceed with the proof of the Equivalence Theorem 2.1 let us discuss the introduced rules and some of their variants. It is difficult to determine where the LOOP II rule was introduced first. We mentioned it in [3]. It also appears in [24, page 64] and in [1, page 151]. The LOOP III rule was introduced in [23]. It corresponds to Dijkstra's modification of his weakest precondition semantics proposed in [11] and reproduced as [12]. We used it in our book [4].

The LOOP IV rule was introduced in [5]. It formalizes the following intuition. In order to prove the termination of a **while** loop it suffices to find a loop invariant and a bound function such that

(i) the loop invariant is maintained by each loop body execution, in the sense of partial correctness,
(ii) each loop body execution decreases the bound function, also in the sense of partial correctness,
(iii) the loop body terminates, and
(iv) the loop invariant implies that the bound function remains non-negative.

This rule supports modular reasoning about program correctness by separating the premises into partial correctness and termination properties. This is of particular relevance in the presence of nested loops, i.e., when the loop body contains inner loops. Then (i) establishes only the partial correctness of the loop body, whereas its termination is relegated to (iii). That the loop body can be iterated only finitely often is established in (ii) in combination with (iv). Note that for loop bodies without inner loops, partial and total correctness coincide. In this case the third premise, $\{p \wedge B\}\ S\ \{\mathbf{true}\}$, can then be dropped, and we arrive at the LOOP III rule. So the hybrid form of this rule arises only for nested loops.

It is easy to manufacture a number of alternative LOOP rules. Note for example that the LOOP II rule results from combining the first two premises of the LOOP III into one. An analogous modification can be carried out in the case of the LOOP IV rule.

Further, the last premise of each of the LOOP rules II-IV and their variants can also be modified, by considering in each case the easier to establish implication $p \wedge B \rightarrow t \geq 0$ instead of $p \rightarrow t \geq 0$. In fact, in [1, page151], [23, page 337], and [24, page 64] this variant of the LOOP II is used. We call the corresponding rules the LOOP B rules.

We shall clarify their proof-theoretic status in Appendix C.

4 Proving Equivalence

A direct proof of the Equivalence Theorem 2.1 would require a detailed analysis of the shape of proofs. For example, in the case of the proof systems II and III we would have to construct in II a proof of the premise $\{p \wedge B \wedge t = z\}\ S\ \{p \wedge t < z\}$ of the LOOP II rule from the proofs of the premises

$$\{p \wedge B\}\ S\ \{p\} \text{ and } \{p \wedge B \wedge t = z\}\ S\ \{t < z\}$$

of the LOOP III rule, which looks daunting.

Therefore we proceed differently. First we introduce a set of auxiliary rules using which we can deduce each LOOP rule from the other and then we show how we get get rid of these auxiliary rules. To explain this approach we introduce first the relevant notions from proof theory.

4.1 Admissibility and Derivability

Assume a given language that is determined by its set of formulas. In what follows we assume that all considered axioms, proof rules, and proof systems are concerned with the same language.

Given a proof system T and two sequences of formulas $\phi_1, \ldots, \phi_m$ and $\varphi_1, \ldots, \varphi_n$ we write

$$\phi_1, \ldots, \phi_m \vdash_T \varphi_1, \ldots, \varphi_n$$

to denote the fact that each formula φ_i can be proved in T using as additional axioms the formulas $\phi_1, \ldots, \phi_m$. We also use this notation when the sequence $\phi_1, \ldots, \phi_m$ is empty and when T is a set of proof rules.

Given a proof system T and a proof rule (in short: a rule) R we abbreviate $T \cup \{R\}$ to $T \cup R$. Given two proof systems T_1 and T_2 we abbreviate the statement that they are equivalent to $T_1 \approx T_2$ and the statement that every formula provable in T_1 is also provable in T_2 to $T_1 \preceq T_2$.

Definition 4.1. (See, e.g. [26, page 47].)

Consider a rule

$$R \qquad \frac{\varphi_1, \ldots, \varphi_k}{\varphi}$$

and a proof system T. R is called

- *admissible in* T if

$$\vdash_T \varphi_1, \ldots, \vdash_T \varphi_k \text{ implies } \vdash_T \varphi,$$

- *derivable in* T if

$$\varphi_1, \ldots, \varphi_k \vdash_T \varphi.$$

So for an admissible rule a proof in T of its conclusion that is manufactured by *manipulating* k proofs in T of its premises can be condensed into *one step*. In turn, a derivable rule allows us to condense a proof in T of its conclusion from its premises into *one step*.

Both types of rules do not increase the power of the proof system T but serve as lemmas that simplify proofs in T by condensing detailed proof arguments into one rule application. Each application of a derivable rule can be expanded locally without affecting the overall proof structure, which in general is not the case with the application of an admissible rule. A derivable rule is also admissible, but in general the converse does not hold.

The following lemma explains the proof-theoretic status of both types of rules and clarifies the difference between them.

Lemma 4.2. *Consider a set of rules AUX.*

(i) Each rule from AUX is admissible in T iff $T \approx T \cup AUX$.

(ii) Each rule from AUX is derivable in T iff for all proof systems U we have $T \cup U \approx T \cup U \cup AUX$.

So derivability is monotone in the sense that if a rule is derivable in T then it is also derivable in any proof system extending T. In constrast, simple examples show that admissibility is not monotone.

Proof. We only prove (ii). Below, when choosing a rule R from AUX, we assume that it has the format

$$\frac{\varphi_1, \ldots, \varphi_k}{\varphi}$$

($\Rightarrow$) Fix a proof system U. We need to show that in every proof in $T \cup U \cup AUX$ each application of a rule from AUX can be eliminated. We proceed by induction on the lengths of proofs. Suppose that in a proof the last rule used was the rule R from AUX. So by the induction hypothesis $\vdash_{T \cup U} \varphi_1, \ldots, \vdash_{T \cup U} \varphi_k$. But R derivable, so $\varphi_1, \ldots, \varphi_k \vdash_{T \cup U} \varphi$ and hence $\vdash_{T \cup U} \varphi$ follows.

($\Leftarrow$) Choose a rule R from AUX. We need to show that $\varphi_1, \ldots, \varphi_k \vdash_T \varphi$, or equivalently $\vdash_{T \cup U} \varphi$, where U consists of the set of axioms $\{\varphi_1, \ldots, \varphi_k\}$. But the latter claim holds since $\vdash_{T \cup U \cup AUX} \varphi$. □

Below we shall need only the left-to-right implications. In what follows we shall use the notion of derivability to analyze similar rules. To make it more visible, given two rules R_1 and R_2 and a set of rules AUX, we shall write 'R_1 can be derived from R_2 using AUX' instead of 'R_1 is derivable in $AUX \cup R_2$'.

The following lemma will allow us to structure the proof of the main result in a natural way.

Lemma 4.3. *Suppose that*

(i) R_1 *can be derived from* R_2 *using* AUX,
(ii) each rule from AUX *is admissible in* $T \cup R_2$.

Then $T \cup R_1 \preceq T \cup R_2$.

Proof. By (i) and Lemma 4.2(ii)

$$T \cup R_2 \cup AUX \approx T \cup R_1 \cup R_2 \cup AUX$$

and by (ii) and Lemma 4.2(i)

$$T \cup R_2 \approx T \cup R_2 \cup AUX.$$

So

$$T \cup R_2 \approx T \cup R_1 \cup R_2 \cup AUX,$$

from which the claim follows since

$$T \cup R_1 \preceq T \cup R_1 \cup R_2 \cup AUX.$$

□

4.2 Auxiliary Rules

The next step consists of identifying the appropriate auxiliary rules. In the proof-theoretic analysis of the LOOP II and III rules, we make use of the following two auxiliary rules, see, e.g., [4].

RULE CONJUNCTION

$$\frac{\{p_1\}\ S\ \{q_1\}, \{p_2\}\ S\ \{q_2\}}{\{p_1 \wedge p_2\}\ S\ \{q_1 \wedge q_2\}}$$

RULE $\exists$-INTRODUCTION

$$\frac{\{p\}\ S\ \{q\}}{\{\exists z_0 : p\}\ S\ \{q\}}$$

where z_0 is a *logical* variable that does not occur in $free(q)$.

To reason about the LOOP IV rule we adopt the following auxiliary rules that relate provability in two proof systems.

RULE PARTIAL CORRECTNESS

$$\frac{\{p\}\ S\ \{q\}}{\vdash_{\mathrm{I}} \{p\}\ S\ \{q\}}$$

RULE HYBRID CONJUNCTION

$$\frac{\vdash_{\mathrm{I}} \{p_1\}\ S\ \{q_1\},\quad \{p_2\}\ S\ \{q_2\}}{\{p_1 \wedge p_2\}\ S\ \{q_1 \wedge q_2\}}$$

This rule generalizes the following rule from [4].

RULE DECOMPOSITION

$$\frac{\vdash_{\mathrm{I}} \{p\}\ S\ \{q\},\quad \{p\}\ S\ \{\mathbf{true}\}}{\{p\}\ S\ \{q\}}$$

Note that each of these auxiliary rules only *adapts* the assertions without changing the program. In the context of Hoare's logic such rules are sometimes called *adaptation rules* [10, 18, 22].

4.3 Structure of the Proof

Equivalence Theorem 2.1 comprises three equivalences, but thanks to the transitivity it suffices to prove two of them. We found it most convenient to deal with the equivalence of the following two pairs of proof systems: (II, III) and (III, IV). To this end we, invoke Lemma 4.3 four times, each time establishing its assumptions for an appropriate pair of LOOP rules R_1 and R_2 and a specific set of rules *AUX* drawn from the set of auxiliary rules listed in the previous section, augmented with the CONSEQUENCE rule (that also is an adaptation rule).

Following Lemma 4.3, we proceed in two steps. Instantiating part (i) of the lemma, we first deal with derivability.

Lemma 4.4 (Derivability). *The following derivability results hold:*

(i) *The* LOOP II *rule can be derived from the* LOOP III *rule, using the* ∃-INTRODUCTION *rule and the* CONSEQUENCE *rule.*

(ii) *The* LOOP III *rule can be derived from the* LOOP II *rule, using the* CONJUNCTION *rule and the* CONSEQUENCE *rule.*

(iii) *The* LOOP IV *rule can be derived from* LOOP III *rule, using the* HYBRID CONJUNCTION *rule and the* CONSEQUENCE *rule.*

(iv) *The* LOOP III *rule can be derived from the* LOOP IV *rule, using the* PARTIAL CORRECTNESS *rule and the* CONSEQUENCE *rule.*

Then, instantiating part (ii) of Lemma 4.3, we deal with admissibilty.

Lemma 4.5 (Admissibility). *The following admissibility results hold:*

(i) *The* ∃-INTRODUCTION *rule is admissible in the proof system* II.

(ii) *The* ∃-INTRODUCTION *rule is admissible in the proof system* III.

(iii) *The* CONJUNCTION *rule is admissible in the proof system* II.

(iv) *The* HYBRID CONJUNCTION *rule is admissible in the proof system* III.

(*v*) *The* PARTIAL CORRECTNESS *rule is admissible in the proof system* IV.

The proofs are given in Appendix B.

To close this discussion note the following consequence of the Equivalence Theorem 2.1 and Lemma 4.2.

Corollary 4.6. *Each* LOOP α *rule is admissible in the proof system* β*, where* $\alpha, \beta \in \{\text{II}, \text{III}, \text{IV}\}$.

Proof. T be the proof system from Appendix A without the LOOP I rule. By the Equivalence Theorem 2.1 for all LOOP rules $\alpha, \beta \in \{\text{II}, \text{III}, \text{IV}\}$ we have $T \cup \alpha \approx T \cup \beta$ and hence $T \cup \beta \approx T \cup \{\alpha, \beta\}$. So by Lemma 4.2(i), the LOOP α rule is admissible in $T \cup \beta$, which is proof system β.. □

4.4 Restrictions on Bound Functions

A careful reader will notice that in the LOOP II, III and IV rules we imposed the restriction $var(t) \subseteq pvar$ on the variables of the bound function t that is absent in the publications that use the LOOP II and III rules. The reason for this restriction is of technical nature: to be able to complete the proofs. For showing in Lemma 4.4 that the LOOP II rule can be derived from the LOOP III rule, we need the auxiliary rule ∃-INTRODUCTION. In turn, in Lemma 4.5 we prove that this auxiliary rule is admissible in the proof system III. This proof is by induction on the structure of **while** programs. The crucial case is that of a **while** loop. By induction hypothesis, we can deduce from

$$\vdash_{\text{III}} \{p_0 \wedge B \wedge t = z\}\ S_0\ \{t < z\}$$

that

$$\vdash_{\text{III}} \{\exists z_0 : (p_0 \wedge B \wedge t = z)\}\ S_0\ \{\exists z_0 : (t < z)\}.$$

Now, to proceed successfully, we wish to deduce further

$$\vdash_{\text{III}} \{(\exists z_0 : p_0) \wedge B \wedge t = z\}\ S_0\ \{t < z\}.$$

This is possible if $z_0 \notin var(B) \cup var(t)$ and $z_0 \neq z$. While the last inequality can be assumed w.l.o.g., the first condition is restrictive. Since z_0 is a logical variable and $var(B) \subseteq pvar$, the imposed restriction $var(t) \subseteq pvar$ makes this reasoning possible.

In a previous version of this paper [5], without the distinction of logical and program variables, we required for the bound function t that $var(t) \subseteq var(B) \cup var(S_0)$ holds. For the ∃-INTRODUCTION rule it was then required that $z_0 \notin var(B) \cup var(S_0)$. So again, $z_0 \notin var(B) \cup var(t)$. However, the condition $var(t) \subseteq var(B) \cup var(S_0)$ is too restrictive for some applications. For example, in [4], we treat in Section 3.9 the case study "Partitioning an Array" and consider a program *PART* that corresponds to the program *Partition* in [14], where Foley and Hoare prove the partial correctness of Hoare's Quicksort algorithm [16]. This program has nested while loops. When proving the termination of the inner loops we needed bound functions that refer to a variable that occurs outside these **while** loops (see p.110 of [4]), so the restriction is not satisfied.

In this paper, we distinguish disjoint sets *lvar* and *pvar* of logical variables and program variables, respectively. A logical variable appears as z to freeze the value of the bound function t. The variables inside programs are all program variables. A bound function t could contain both logical and program variables. Our requirement $var(t) \subseteq pvar$ is therefore a restriction. It enables the proof of Lemma 4.5, as explained above. The restriction is weak enough for performing the termination proof of the partitioning program *PART*.

What about a LOOP rule without any restriction on the bound function t ? To prove the Equivalence Theorem 2.1 in this setting ideally we would like to reason at the level of proof rules, using an appropriate substitution rule. However, we did not succeed in establishing admissibility of such a rule. So instead we reason at the level of proofs.

Consider a proof of a correctness formula ϕ in a proof system $\alpha \in \{\text{II}, \text{III}, \text{IV}\}$ in which no distinction is made between the logical and program variables and in which in the corresponding LOOP rule no restriction is imposed on the bound function t. First, replace all occurrences of the z variables by distinct fresh variables that do not occur in the considered program or in the used bound functions t. Clearly, this remains a proof of ϕ in α.

Next, let the set of logical variables *lvar* consist of these fresh variables and let *pvar* be the set of remaining variables. Then all applications of the corresponding LOOP rule obey the restriction used in this paper. So by the Equivalence Theorem 2.1 the correctness formula ϕ can also be proved in any other proof system $\beta \in \{\text{II}, \text{III}, \text{IV}\}$ and hence, a fortiori in the proof system β in which in the corresponding LOOP rule no restriction on the bound function t is used.

5 Representing Proofs

5.1 Proof outlines

Even though the LOOP rules for proving termination are equivalent in the sense of the Equivalence Theorem 2.1, they lead to different proofs of total correctness of **while** programs. The idea behind the LOOP III rule is to establish that p is a loop invariant and t is a bound function separately, while in the LOOP II rule both facts are established simultaneously. So the LOOP III rule looks more convenient when we want to strengthen a proof of partial correctness to a proof of total correctness: it suffices to establish two new premises concerned with the bound function t. In turn, the LOOP IV rule allows us to split the proof obligations even further, by identifying the property actually needed to be proved in terms of total correctness. This, as already mentioned, allows one to support modular reasoning.

However, matters change when we want to represent the proofs in the resulting proof systems in a convenient form. Given that these proofs deal with structured programs, their most natural representation consists of so-called ***proof outlines***, a notion introduced in [23]. Informally, it is a proof representation in the form of a program annotated by the assertions arising from the appropriate rule applications. Such a representation is possible thanks to the fact that the proof rules are syntax directed. Proof outlines were introduced in [23] in order to reason about correctness of parallel programs, where they served to establish so-called interference freedom among the proofs of the component programs. However, they are also very useful as a representation of correctness proofs of sequential programs and, when some obvious assertions are deleted, as a program documentation.

Now, given that the LOOP III and IV rules have more than one premise consisting of a correctness formula, it is difficult to employ a single proof outline to represent a proof involving any of these rules. This is not the case with the LOOP II rule. To illustrate this point recall first that the proof outlines are defined by induction on the program structure. We only focus on the crucial formation rule concerned with the **while** statement. For the proof system I the following formation rule was used in [4]:

$$\frac{\{p \wedge B\}\ S^*\ \{p\}}{\{\mathbf{inv} : p\}\ \mathbf{while}\ B\ \mathbf{do}\ \{p \wedge B\}\ S^*\ \{p\}\ \mathbf{od}\ \{p \wedge \neg B\}}$$

where S^* is the program S annotated with some assertions.

For the proof system II we introduce the following formation rule:

$$\frac{\{p \wedge B \wedge t = z\}\ S^*\ \{p \wedge t < z\},\quad p \rightarrow t \geq 0}{\{\mathbf{inv} : p\}\{\mathbf{bd} : t\}\ \mathbf{while}\ B\ \mathbf{do}\ \{p \wedge B \wedge t = z\}\ S^*\ \{p \wedge t < z\}\ \mathbf{od}\ \{p \wedge \neg B\}}$$

where S^* and S^{**} are annotations of the program S with some assertions, t is an integer expression and z is an integer variable not occurring in p, t, B or S^*.[3]

For a moment we defer the discussion of proof outlines for the proof system IV. One can easily prove by induction that each proof outline for the proof system I corresponds to a proof in this proof system. For example, if by the induction hypothesis the proof outline $\{p \wedge B\}\ S^*\ \{p\}$ corresponds to a proof of $\{p \wedge B\}\ S\ \{p\}$ in I, then the proof outline

$$\{\mathbf{inv} : p\}\ \mathbf{while}\ B\ \mathbf{do}\ \{p \wedge B\}\ S^*\ \{p\}\ \mathbf{od}\ \{p \wedge \neg B\}$$

corresponds to a proof of $\{p\}$ **while** B **do** S **od** $\{p \wedge \neg B\}$ in I, which is obtained by applying to $\{p \wedge B\}\ S\ \{p\}$ the LOOP I rule.

However, this property fails to hold for the proof outlines for the proof system III, because the second proof outline used in the premise of the formation rule is dropped. As a consequence, the proof outline for the **while** statement does not allow one to reconstruct the proof of $\{p\}$ **while** B **do** S **od** $\{p \wedge \neg B\}$ in the proof system III.

5.2 Proofs using the LOOP II rule

By contrast, each proof outline for the proof system II involving the LOOP II rule does correspond to a proof in this proof system, because all assertions used are retained. In other words, from each proof outline for the proof system II a proof in this system can be extracted.

To illustrate this point consider the following program S_N involving nested loops, suggested to us by Tobias Nipkow (private communication):

```
S_N ≡ while i < n do
          j := i;
          while 0 < j do
              j := j − 1
          od;
          i := i + 1
      od
```

where i, j, n are integer variables.

We would like to prove that it terminates for all initial states. To this end, we prove the correctness formula $\{\mathbf{true}\}\ S_N\ \{\mathbf{true}\}$ in the proof system II. In Fig. 1, we show the proof in the form of a proof outline, instantiating the corresponding formation rule for the **while** statement in proof system II with the following loop invariants and bound functions for the outer and the inner loop, respectively:

$$p \equiv \mathbf{true} \text{ and } t \equiv max(n - i, 0),$$

$$p \equiv n - i = z_1 \wedge z_1 > 0 \text{ and } t \equiv max(j, 0).$$

[3] In [4], in contrast to [3], there is a typo and S^{**} is mentioned here instead of S^*.

Note that in a proof outline adjacent assertions stand for implications according to an application of the CONSEQUENCE rule. For instance, the assertion in line 3 implies that of line 4. Assignments are treated by backward substitution according to the ASSIGNMENT axiom. For instance, the assignment $i := i + 1$ in line 18 is dealt with by substituting i by $i + 1$ in the assertion in line 19, yielding the assertion in line 17.

1 $\{\mathbf{inv} : \mathbf{true}\}\ \{\mathbf{bd} : max(n - i, 0)\}$
2 **while** $i < n$ **do**
3 $\quad\{\mathbf{true} \wedge i < n \wedge max(n - i, 0) = z_1\}$
4 $\quad\{n - i = z_1 \wedge z_1 > 0\}$
5 $\quad j := i;$
6 $\quad\{n - i = z_1 \wedge z_1 > 0\}$
7 $\quad\{\mathbf{inv} : n - i = z_1 \wedge z_1 > 0\}\ \{\mathbf{bd} : max(j, 0)\}$
8 $\quad$**while** $0 < j$ **do**
9 $\quad\quad\{n - i = z_1 \wedge z_1 > 0 \wedge 0 < j \wedge max(j, 0) = z_2\}$
10 $\quad\quad\{n - i = z_1 \wedge z_1 > 0 \wedge j = z_2 \wedge z_2 > 0\}$
11 $\quad\quad\{n - i = z_1 \wedge z_1 > 0 \wedge j - 1 < z_2 \wedge z_2 > 0\}$
12 $\quad\quad j := j - 1$
13 $\quad\quad\{n - i = z_1 \wedge z_1 > 0 \wedge j < z_2 \wedge z_2 > 0\}$
14 $\quad\quad\{n - i = z_1 \wedge z_1 > 0 \wedge max(j, 0) < z_2\}$
15 $\quad$**od**;
16 $\quad\{n - i = z_1 \wedge z_1 > 0 \wedge \neg(0 < j)\}$
17 $\quad\{n - (i + 1) < z_1 \wedge z_1 > 0\}$
18 $\quad i := i + 1$
19 $\quad\{n - i < z_1 \wedge z_1 > 0\}$
20 $\quad\{\mathbf{true} \wedge max(n - i, 0) < z_1\}$
21 **od**
22 $\{\mathbf{true} \wedge \neg(i < n)\}$
23 $\{\mathbf{true}\}$

Fig. 1. Proof outline for $\{\mathbf{true}\}\ S_N\ \{\mathbf{true}\}$ in the proof system II. The line numbers have been added for reference only.

5.3 Proofs using the LOOP IV rule

Let us now move on to a discussion of the proofs involving the LOOP IV rule. This rule, just like the LOOP III rule, uses more than one correctness formula as a premise. As a result, it shares with the LOOP III rule the problem that it is not clear how to faithfully represent correctness proofs using a single proof outline. Indeed, each of the first three premises calls for a separate proof outline.

But it is not difficult to see that this would give rise to largely overlapping proof outlines. So, instead, we propose an alternative approach in which we replace these overlapping proof outlines referring to the proof system IV by an interrelated *set* of proof outlines in the sense of *partial correctness*, so referring to the proof system I.

For a given correctness formula $\{p\}\ S\ \{q\}$ to be proved in the proof system IV this set is defined as follows. First, we have a proof outline $\{p\}\ S^*\ \{q\}$ that employs the first formation rule given above and thus represents a proof of partial correctness of $\{p\}\ S\ \{q\}$ in I.

Next, for each occurrence of a loop **while** B **do** S_0 **od** in S, we have a proof outline $\{p_0 \wedge B \wedge t = z\}\ S_0^*\ \{t < z\}$, which represents a proof of partial correctness of $\{p_0 \wedge B \wedge t = z\}\ S_0\ \{t < z\}$ in I. Here p_0 is the invariant associated with the occurrence of the loop **while** B **do** S_0 **od** in the above proof outline $\{p\}\ S^*\ \{q\}$ and t is some bound function t

such that $p_0 \rightarrow t \geq 0$. We omit the proof that existence of such a set of proof outlines in the sense of partial correctness ensures a proof of the corresponding correctness formula in the proof system IV. Intuitively, the use of the above proof outlines for *each* loop occurrence in S ensures by structural induction the third premise of the LOOP IV rule, so $\{p \wedge B\}\ S\ \{\mathbf{true}\}$ in the sense of total correctness.

We illustrate how such a set of proof outlines can be used to establish the proof of the correctness formula $\{\mathbf{true}\}\ S_N\ \{\mathbf{true}\}$ for Nipkow's program S_N in the proof system IV. We skip the trivial proof outline $\{\mathbf{true}\}\ S_N^*\ \{\mathbf{true}\}$, which corresponds to a proof of partial correctness of $\{\mathbf{true}\}\ S_N\ \{\mathbf{true}\}$. Let S_0 denote the body of the outer loop of S_N. Given the bound function $max(n-i, 0)$, Fig. 2 shows the proof outline which corresponds to a proof of partial correctness of

$$\{\mathbf{true} \wedge i < n \wedge max(n-i,0) = z\}\ S_0\ \{max(n-i,0) < z\},$$

assuming that the trivial invariant **true** is associated with this loop in the proof outline $\{\mathbf{true}\}\ S_N^*\ \{\mathbf{true}\}$. Note that $\mathbf{true} \rightarrow max(n-i,0) \geq 0$.

$\{\mathbf{true} \wedge i < n \wedge max(n-i,0) = z\}$
$\{n - i = z \wedge z > 0\}$
  $j := i;$
$\{n - i = z \wedge z > 0\}$
$\{\mathbf{inv} : n - i = z \wedge z > 0\}$
while $0 < j$ **do**
    $\{n - i = z \wedge z > 0\}$
      $j := j - 1$
    $\{n - i = z \wedge z > 0\}$
od;
$\{n - i = z \wedge z > 0 \wedge \neg(0 < j)\}$
$\{n - (i+1) < z \wedge z > 0\}$
  $i := i + 1$
$\{n - i < z \wedge z > 0\}$
$\{max(n-i,0) < z\}$

Fig. 2. Proof outline for $\{\mathbf{true} \wedge i < n \wedge max(n-i,0) = z\}\ S_0\ \{max(n-i,0) < z\}$ in the proof system I.

Finally, the LOOP IV rule requires to prove termination of S_0. This boils down to establish the termination of the inner loop. To this end, we introduce the bound function $max(j, 0)$. Note that $\mathbf{true} \rightarrow max(j,0) \geq 0$. Fig. 3 shows the proof outline which corresponds to a proof of partial correctness of

$$\{\mathbf{true} \wedge 0 < j \wedge max(j,0) = z\}\ j := j - 1\ \{max(j,0) < z\}.$$

6 Practical Applications

Research on program verification has entered practice most visibly by the use of assertions as annotations of programs and program interfaces that remain to be implemented. This can be seen in the paradigm of *design by contract* introduced by Bertrand Meyer for his object-oriented programming language Eiffel [21]: program design starts with a specification in terms of assertions, the contract, against which the program is to be checked either statically by means of a proof or dynamically at runtime.

$\{\mathbf{true} \wedge 0 < j \wedge max(j, 0) = z\}$
$\{j - 1 < z \wedge z > 0\}$
$j := j - 1$
$\{j < z \wedge z > 0\}$
$\{max(j, 0) < z\}$

Fig. 3. Proof outline for $\{\mathbf{true} \wedge 0 < j \wedge max(j, 0) = z\}\ j := j - 1\ \{max(j, 0) < z\}$ in the proof system I.

This paradigm has been adopted and extended to other programming languages, in particular to Java. The *Java Modeling Language* (JML) enriches Java with facilities for writing assertions (pre- and postconditions as well as class invariants) but also with a concept of abstract state space (using so-called model variables) [19][4]. For assertions, JML uses Java's Boolean expressions extended by universal and existential quantifiers. They are directly written into the Java source code in the form of comments starting with the symbols `//@` (so that the annotated source code may be processed by both an ordinary Java compiler and a specialized JML tool).

JML is designed to deal with the specification of Java classes, but here we focus on loops. JML provides the designated keywords `requires` for specifying the precondition, `ensures` for the postcondition, `loop_invariant`, and `loop_decreases` for the bound function. To enhance readability, the pre- and postcondition as well as the loop invariant may be split into several assertions, each one stated after a separate repeated keyword[5]. An annotated Java program corresponds to a proof outline as discussed here, but restricted to the essential assertions for each loop (pre- and postcondition, loop invariant and bound function).

Also for the programming language C, a standardized specification language for C programs, called ACSL and inspired by JML, has been designed, see for instance [8][6].

In this paper, we have shown that the rules LOOP II-IV for proving termination of loops are equivalent. With respect to the number of premises LOOP II is clearly the simplest rule for proving termination. However, a proof using this rule in general require more complex assertions because of the accumulation of the specifications of the bound functions of inner loops. This can be seen in the proof outline given in Fig. 1, in which the assertions inside the inner loop refer to both z_1 and z_2, where z_1 is the variable freezing the value of the bound function $max(n - i, 0)$ of the outer loop and z_2 is the variable freezing the value of the bound function $max(j, 0)$ of the inner loop. The reason is that in this proof outline we need to establish that the value of the bound function of the outer loop is not affected by the inner loop and that the value of the other bound function decreases.

The LOOP IV rule allows for a separate proof of termination of each loop, which does not require the specification of bound functions for the inner loops. Thus we have a trade off between a single complex proof and a number of simpler proofs, where complexity is measured by the size of the assertions. What works best in practice depends on the particular program structure.

7 Proofs from Assumptions

Admittedly, the equivalence of the proof systems II-IV is a limited result, since it concerns only the class of **while** programs. Here we show how to extend the Equivalence Theorem 2.1 to a more realistic class of programs.

[4]See also `https://www.cs.ucf.edu/~leavens/JML/index.shtml`
[5]For an example, see `https://www.openjml.org/examples/binary-search.html`
[6]See also `https://frama-c.com/download/acsl-1.20.pdf`

To this end, consider a situation in which programs can access a library of procedures each of which is specified by means of a correctness formula. The access is provided by means of procedure calls. For simplicity we assume the parameter mechanism *call-by-value*, according to which the values of the fomal parameters are assigned to the formal parameters.

Let us start with a motivating toy example. Consider the following program that invokes an unknown (though of course one with the expected meaning) procedure *swap(x,y)*:

$$GCD \equiv \textbf{while } x \neq q \textbf{ do if } x < y \textbf{ then } swap(x, y) \textbf{ fi}; x := x - y \textbf{ od}.$$

We use in it the statement **if** B **then** S **fi** with the corresponding proof rule

RULE IF

$$\frac{\{p \wedge B\}\ S\ \{q\},\ p \wedge \neg B \rightarrow q}{\{p\}\ \textbf{if } B \textbf{ then } S \textbf{ fi}\ \{q\}}$$

We want to prove

$$\begin{aligned} &\{x = x_0 \wedge y = y_0\}\ swap(x, y)\ \{y = x_0 \wedge x = y_0\} \vdash \\ &\{x = x_0 \wedge y = y_0 \wedge x > 0 \wedge y > 0\}\ GCD\ \{x = y = gcd(x_0, y_0\}, \end{aligned} \tag{1}$$

where x_0 and y_0 are logical variables and x and y are program variables. That is, we want to establish total correctness of the *GCD* program under the stated assumption about the procedure call *swap(x,y)*. It is clear how to prove the above claim in any of the proof systems II-IV: in the proof we can simply use the assumed property of *swap(x,y)* as an additional axiom. However, care has to be exercised because some auxiliary rules are needed.

The reason is that we need first to derive a more general property of *swap(x,y)*, namely the correctness formula $\{p(x, y\}\ swap(x, y)\ \{p(y, x)\}$, where p is an assertion with free variables including x and y and which we shall use for a specific p. To establish it we need the following auxiliary axiom *for partial correctness* (see [4, page 98]):

AXIOM INVARIANCE

$$\vdash_{\mathrm{I}} \{r\}\ S\ \{r\}$$

where no variable in $free(r)$ can be changed in S.

Using it for the assertion $p(x_0, y_0)$ together with the assumption

$$\{x = x_0 \wedge y = y_0\}\ swap(x, y)\ \{y = x_0 \wedge x = y_0\}$$

we obtain first by the HYBRID CONJUNCTION rule

$$\{p(x_0, y_0) \wedge x = x_0 \wedge y = y_0\}\ swap(x, y)\ \{p(x_0, y_0) \wedge y = x_0 \wedge x = y_0\},$$

from which by the CONSEQUENCE rule and two applications of the $\exists$-INTRODUCTION rule (we assume here that x_0 and y_0 are logical variables) we get

$$\{\exists x_0, y_0 : (p(x_0, y_0) \wedge x = x_0 \wedge y = y_0)\}\ swap(x, y)\ \{p(y, x)\},$$

from which by the CONSEQUENCE rule $\{p(x, y\}\ swap(x, y)\ \{p(y, x)\}$ follows.

Then we can exhibit an appropriate proof outline in the proof system II that at the relevant place uses the just established property of *swap(x,y)* for

$$p \equiv gcd(x, y) = gcd(x_0, y_0) \wedge x > 0 \wedge y > 0 \wedge z = x + y \wedge x < y.$$

$\{x = x_0 \wedge y = y_0 \wedge x > 0 \wedge y > 0\}$
$\{gcd(x, y) = gcd(x_0, y_0) \wedge x > 0 \wedge y > 0\}$
$\{\mathbf{inv} : gcd(x, y) = gcd(x_0, y_0) \wedge x > 0 \wedge y > 0\}$ $\{\mathbf{bd} : x + y\}$
while $x \neq y$ **do**
  $\{gcd(x, y) = gcd(x_0, y_0) \wedge x > 0 \wedge y > 0 \wedge z = x + y \wedge x \neq y\}$
  if $x < y$ **then**
    $\{gcd(x, y) = gcd(x_0, y_0) \wedge x > 0 \wedge y > 0 \wedge z = x + y \wedge x < y\}$
    $swap(x, y)$
    $\{gcd(y, x) = gcd(x_0, y_0) \wedge y > 0 \wedge x > 0 \wedge z = y + x \wedge y < x\}$
    $\{gcd(x, y) = gcd(x_0, y_0) \wedge x > 0 \wedge y > 0 \wedge z = x + y \wedge x > y\}$
  fi;
  $\{gcd(x, y) = gcd(x_0, y_0) \wedge x > 0 \wedge y > 0 \wedge z = x + y \wedge x > y\}$
  $x := x - y$
  $\{gcd(x, y) = gcd(x_0, y_0) \wedge x > 0 \wedge y > 0 \wedge z < x + y\}$
od
$\{gcd(x, y) = gcd(x_0, y_0) \wedge x > 0 \wedge y > 0 \wedge x = y\}$
$\{x = y = gcd(x, y) \wedge gcd(x, y) = gcd(x_0, y_0)\}$
$\{x = y = gcd(x_0, y_0)\}$

Fig. 4. Proof outline for the conditional correctness formula (1).

To extend our study of the LOOP rules to such a setting we begin by allowing uninterpreted procedure calls, so program statements of the form $P(\bar{t})$. This leads to a larger class of programs to which we are referring below. Next, we consider *conditional correctness formulas*, which are constructs of the form

$$\phi_1, \ldots, \phi_n \vdash \{p\}\ S\ \{q\},$$

where $n \geq 0$, each ϕ_i is an assertion or a correctness formula referring to a procedure call and S is a program that may contain any of the procedure calls appearing in some ϕ_i.

It is clear how to modify all considered proof rules to deal with the conditional correctness formulas: it suffices to prefix each correctness formula by '$\phi_1, \ldots, \phi_n \vdash$'.

Additionally, we add the following axiom:

SELECT AXIOM

$$\phi_1, \ldots, \phi_n \vdash \phi_i,$$

where $i \in \{1, \ldots, n\}$.

This yields the proof systems CI-CIV. Optionally, we could add an appropriate rule that allows us to derive from assumptions about generic procedure calls adjusted assumptions about procedure calls with actual parameters. For details see the INSTANTIATION rule in [4, page 159] which lists the appropriate conditions on formal and actual parameters.

Note that for the **while** programs the INVARIANCE axiom is admissible in the proof system I, that is the following observation holds.

Note 1. *For every **while** program S we have $\vdash_{\mathrm{I}} \{r\}\ S\ \{r\}$, where no variable in free(r) can be changed in S.*

However, this axiom cannot be dispensed with for uninterpreted procedure calls, which explains why we need to retain it.

We would like to extend the Equivalence Theorem 2.1 to the proof systems CI-CIV, that is to prove their equivalence. However, such a result does not hold. To see this consider the following example. For a procedure call S we clearly can prove in CIII

$$\{p \wedge B\}\ S\ \{p\},\ \{p \wedge B \wedge t = z\}\ S\ \{t < z\},\ p \rightarrow t \geq 0 \vdash$$
$$\{p\}\ \textbf{while}\ B\ \textbf{do}\ S\ \textbf{od}\ \{p \wedge \neg B\}.$$

Indeed, it just suffices to apply the LOOP III rule. However, this conditional correctness formula cannot be proved in CII because in this proof system there is no way to prove the premise

$$\{p \wedge B\}\ S\ \{p\},\ \{p \wedge B \wedge t = z\}\ S\ \{t < z\},\ p \rightarrow t \geq 0 \vdash$$
$$\{p \wedge B \wedge t = z\}\ S\ \{p \wedge t < z\}$$

of the LOOP II rule.

It is worthwhile to note that this premise can be directly proved using the CONJUNCTION rule. Now, the Admissibility Lemma 4.5(iii) states that this rule is admissible in the proof system II, so one might think that such an application of this rule can be eliminated. However, in this extended setting this cannot be done, because we deal with uninterpreted procedure calls. More specifically, the extended correctness formula

$$\{p_1\}\ S\ \{q_1\}, \{p_2\}\ S\ \{q_2\} \vdash \{p_1 \wedge p_2\}\ S\ \{q_1 \wedge q_2\}$$

can be proved by applying first the SELECT axiom twice to get

$$\{p_1\}\ S\ \{q_1\}, \{p_2\}\ S\ \{q_2\} \vdash \{p_1\}\ S\ \{q_1\}$$

and

$$\{p_1\}\ S\ \{q_1\}, \{p_2\}\ S\ \{q_2\} \vdash \{p_2\}\ S\ \{q_2\}$$

and then by applying the CONJUNCTION rule. If this last rule application could be eliminated we would establish that the CONJUNCTION rule is derivable in the proof system II, which is not the case.

Consequently, only the following counterpart of the Equivalence Theorem 2.1 holds, where AUX denotes the set of auxiliary rules introduced in Subection 4.2. To prove the Equivalence Theorem 2.1, we also use auxiliary rule in the Derivability Lemma 4.4 and then show that these auxiliary rules are admissible in Lemma 4.5. Why does this two-step approach not work here as well ?

Theorem 7.1. *The proof systems CII $\cup AUX$ – CIV $\cup AUX$ are equivalent.*

Proof. We only need to deal with the applications of the LOOP rules. To this end it is sufficient to invoke a straightforward modification of the Derivability Lemma 4.4 to the conditional correctness formulas obtained by prefixing each correctness formula by '$\phi_1, \ldots, \phi_n \vdash$'. By items (i) and (ii) of this lemma we get that CII $\cup AUX$ and CIII $\cup AUX$ are equivalent and by items (iii) and (iv) we get that CIII $\cup AUX$ and CIV $\cup AUX$ are equivalent. This concludes the proof. □

This result underlies, yet again, the importance of auxiliary rules in this comparative study of the LOOP rules.

8 Conclusion and Future Work

We provided here a comparative proof-theoretic analysis of three proof rules for proving termination of **while** programs and their variants. By introducing the distinction between the admissible and derivable proof rules, we could clarify the crucial role auxiliary rules play in the proof of the corresponding equivalence theorem. More specifically, we showed that each loop rule for termination can be derived from any other one using certain auxiliary rules that subsequently can be dispensed with by showing that they are admissible. We also explained that in spite of this equivalence result the proof representations in the form of proof outlines differ, and discussed what consequence this observation has in the context of the paradigm of design by contract.

Further, the distinction between logical and program variables (and subsequent removal of them) allowed us to deal with a restriction (on the bound functions) that we needed to impose to obtain the appropriate equivalence result in [5]. Additionally, we showed how to extend these results to the setting in which programs have access to a library of procedures. It turns out that in this setting the auxiliary rules cannot be dispensed with.

Under current investigation is an extension of our comparative analysis to the following rule which has been introduced in [7] to deal with *unbounded* nondeterminism.

$$\frac{\begin{array}{l} p(x) \wedge x > 0 \rightarrow B, \\ \{p(x) \wedge x > 0\}\ S\ \{\exists y : p(y) \wedge y < x\}, \\ p(0) \rightarrow \neg B, \end{array}}{\{\exists x : p(x)\}\ \textbf{while}\ B\ \textbf{do}\ S\ \textbf{od}\ \{p(0)\}}$$

In this rule it was stipulated that the parameter x ranges over ordinals instead over integers, but in the context of programs which only give rise to *bounded* nondeterminism the range of integers suffices. Parameterized LOOP invariants were first used in a rule proposed in [15].

Further, the question naturally arises to what extent the separation of concerns underlying the different LOOP rules is applicable to proving termination of recursive procedures.

References

[1] J. B. Almeida, M. J. Frade, J. S. Pinto, and S. Melo de Sousa. 2011. *Rigorous Software Development: An Introduction to Program Verification.* Springer, London.

[2] K. R. Apt. 1981. Recursive Assertions and Parallel Programs. *Acta Inf.* 13 (1981), 219–232. https://doi.org/10.1007/BF00289262

[3] K. R. Apt, F. S. de Boer, and E.-R. Olderog. 1990. Proving termination of parallel programs. In *Beauty is Our Business, A Birthday Salute to Edsger W. Dijkstra*, W. H. J. Feijen, A. J. M. van Gasteren, D. Gries, and J. Misra (Eds.). Springer-Verlag, New York, 0–6.

[4] K. R. Apt, F. S. de Boer, and E.-R. Olderog. 2009. *Verification of Sequential and Concurrent Programs* (third ed.). Springer-Verlag, New York. https://doi.org/10.1007/978-1-84882-745-5

[5] K. R. Apt, F. S. de Boer, and E.-R. Olderog. 2025. Three Ways of Proving Termination of Loops. In *The Combined Power of Research, Education, and Dissemination - Essays Dedicated to Tiziana Margaria on the Occasion of Her 60th Birthday (LNCS, Vol. 15240)*, Mike Hinchey and Bernhard Steffen (Eds.). Springer, Cham, 280–301. https://doi.org/10.1007/978-3-031-73887-6_19

[6] K. R. Apt and E.-R. Olderog. 2019. Fifty years of Hoare's logic. *Formal Aspects of Computing* 31, 6 (2019), 751–807. https://doi.org/10.1007/S00165-019-00501-3

[7] K. R. Apt and G. D. Plotkin. 1986. Countable nondeterminism and random assignment. *J. ACM* 33, 4 (1986), 724–767.

[8] P. Baudin, F. Bobot, D. Bühler, L. Correnson, F. Kirchner, N. Kosmatov, A. Maroneze, V. Perrelle, V. Prevosto, J. Signoles, and N. Williams. 2021. The dogged pursuit of bug-free C programs: the Frama-C software analysis platform. *Commun. ACM* 64, 8 (2021), 56–68. https://doi.org/10.1145/3470569

[9] B. Cook, A. Podelski, and A. Rybalchenko. 2011. Proving program termination. *Commun. ACM* 54, 5 (2011), 88–98. https://doi.org/10.1145/1941487.1941509

[10] S. de Gouw and J. Rot. 2016. Effectively Eliminating Auxiliaries. In *Theory and Practice of Formal Methods – Essays Dedicated to Frank de Boer on the Occasion of His 60th Birthday (LNCS, Vol. 9660)*, E. Ábrahám, M. M. Bonsangue, and E. B. Johnsen (Eds.). Springer, Cham, 226–241. https://doi.org/10.1007/978-3-319-30734-3_16

[11] E. W. Dijkstra. 1976. A great improvement. (July 1976). Available at http://www.cs.utexas.edu/users/EWD/ewd05xx/EWD573.PDF, published as [12].

[12] E. W. Dijkstra. 1982. A great improvement. In *Selected Writings on Computing: A Personal Perspective*. Springer-Verlag, New York, Heidelberg, Berlin, 217–219.

[13] R. W. Floyd. 1967. Assigning Meaning to Programs. In *Proceedings of Symposium on Applied Mathematics 19, Mathematical Aspects of Computer Science*, J. T. Schwartz (Ed.). American Mathematical Society, New York, 19–32.

[14] M. Foley and C. A. R. Hoare. 1971. Proof of a Recursive Program: Quicksort. *Comput. J.* 14, 4 (1971), 391–395. https://doi.org/10.1093/COMJNL/14.4.391

[15] D. Harel. 1979. *First-Order Dynamic Logic*. LNCS, Vol. 68. Springer, Berlin, Heidelberg. https://doi.org/10.1007/3-540-09237-4

[16] C. A. R. Hoare. 1961. Algorithm 64: Quicksort. *Commun. ACM* 4, 7 (1961), 321. https://doi.org/10.1145/366622.366644

[17] C. A. R. Hoare. 1969. An Axiomatic Basis for Computer Programming. *Commun. ACM* 12 (1969), 576–580, 583. https://doi.org/10.1145/363235.363259

[18] C. A. R. Hoare. 1971. Procedures and parameters: an axiomatic approach. In *Proceedings of Symposium on the Semantics of Algorithmic Languages (Lecture Notes in Mathematics, Vol. 188)*, E. Engeler (Ed.). Springer-Verlag, New York, 102–116. https://doi.org/10.1007/BFB0059696

[19] G. T. Leavens, Y. Cheon, C. Clifton, C. Ruby, and D. R. Cok. 2005. How the Design of JML Accomodates Both Runtime Assertion Checking and Formal Verification. *Sci. of Comput. Prog.* 55 (2005), 185–208. https://doi.org/10.1016/J.SCICO.2004.05.015

[20] Z. Manna and A. Pnueli. 1974. Axiomatic approach to total correctness of programs. *Acta Inf.* 3 (1974), 253–263. https://doi.org/10.1007/BF00288637

[21] B. Meyer. 1997. *Object-Oriented Software Construction* (2nd ed.). Prentice Hall, New Jersey. Available at https://bertrandmeyer.com/wp-content/upLoads/OOSC2.pdf.

[22] E.-R. Olderog. 1983. On the notion of expressiveness and the rule of adaptation. *Theoretical Computer Science* 30 (1983), 337–347. https://doi.org/10.1016/0304-3975(83)90009-9

[23] S. Owicki and D. Gries. 1976. An Axiomatic Proof Technique for Parallel Programs. *Acta Inf.* 6 (1976), 319–340. https://doi.org/10.1007/BF00268134

[24] J. C. Reynolds. 1998. *Theories of Programming Languages*. Cambridge University Press, Cambridge, Great Britain.

[25] J. Tiuryn. 2002. Hoare Logic: From First-Order to Propositional Formalism. In *Proof and System-Reliability*, H. Schwichtenberg and R. Steinbrüggen (Eds.). Springer, Dordrecht, 323–340. https://doi.org/10.1007/978-94-010-0413-8_10

[26] J. von Plato. 2014. *Elements of Logical Reasoning*. Cambridge University Press, Cambridge, Great Britain. https://doi.org/10.1017/CBO9781139567862

A Proof System I

The proof system I consists of the following axiom and rules:

AXIOM ASSIGNMENT

$$\{p[u := t]\}\ u := t\ \{p\}$$

RULE COMPOSITION

$$\frac{\{p\}\ S_1\ \{r\}, \{r\}\ S_2\ \{q\}}{\{p\}\ S_1;\ S_2\ \{q\}}$$

RULE CONDITIONAL

$$\frac{\{p \wedge B\}\ S_1\ \{q\}, \{p \wedge \neg B\}\ S_2\ \{q\}}{\{p\}\ \textbf{if}\ B\ \textbf{then}\ S_1\ \textbf{else}\ S_2\ \textbf{fi}\ \{q\}}$$

RULE LOOP I

$$\frac{\{p \wedge B\}\ S\ \{p\}}{\{p\}\ \textbf{while}\ B\ \textbf{do}\ S\ \textbf{od}\ \{p \wedge \neg B\}}$$

RULE CONSEQUENCE

$$\frac{p \rightarrow p_1, \{p_1\}\ S\ \{q_1\}, q_1 \rightarrow q}{\{p\}\ S\ \{q\}}$$

Additionally, given an interpretation $\mathcal{I}$ for the underlying first-order language, we use as axioms all assertions that are true in $\mathcal{I}$. These assertions are used as premises in the CONSEQUENCE rule.

B Proof of the Equivalence Theorem 2.1

As explained in Subsection 4.3, to prove the Equivalence Theorem 2.1 (The proof systems II–V are equivalent.), it suffices to instantiate Lemma 4.3, for each of the four pair of rules

$$(R_1, R_2) \in \{(\mathrm{II}, \mathrm{III}), (\mathrm{III}, \mathrm{II}), (\mathrm{III}, \mathrm{IV}), (\mathrm{IV}, \mathrm{III})\}$$

and each time for an appropriate set of rules AUX.

B.1 Derivability of the LOOP Rules

We provide the proof of the Derivability Lemma 4.4, repeating each time the statement to be proved.

Proof. The proofs have the same shape given that the last premise of all LOOP rules is the same.
(i) The LOOP II rule can be derived from the LOOP III rule, using the ∃-INTRODUCTION rule and the CONSEQUENCE rule.

It suffices to show how to derive from the premise

$$\{p \wedge B \wedge t = z\}\ S\ \{p \wedge t < z\}$$

of the LOOP II rule the first two premises of the LOOP III rule. We first derive by the CONSEQUENCE rule both

$$\{p \wedge B \wedge t = z\}\ S\ \{p\} \quad \text{and} \quad \{p \wedge B \wedge t = z\}\ S\ \{t < z\},$$

which is the second premise of the LOOP III rule.

Next, by the ∃-INTRODUCTION rule, we derive

$$\{\exists z : (p \wedge B \wedge t = z)\}\ S\ \{p\}.$$

from $\{p \wedge B \wedge t = z\}\ S\ \{p\}$.

By assumption $var(t) \subseteq pvar$ and z does not appear in p. Also $var(B) \subseteq pvar$, so z, as a logical variable, does not appear in p, B or t. Hence

$$p \wedge B \rightarrow (p \wedge B \wedge \exists z : t = z) \rightarrow \exists z : (p \wedge B \wedge t = z).$$

So by the CONSEQUENCE rule we derive also the first premise $\{p \wedge B\}\ S\ \{p\}$ of the LOOP III rule.

(ii) The LOOP III rule can be derived from the LOOP II rule, using the CONJUNCTION rule and the CONSEQUENCE rule.

It suffices to note that from the first two premises

$$\{p \wedge B\}\ S\ \{p\} \quad \text{and} \quad \{p \wedge B \wedge t = z\}\ S\ \{t < z\}$$

of the LOOP III rule we can derive the first premise

$$\{p \wedge B \wedge t = z\}\ S\ \{p \wedge t < z\}$$

of the LOOP II rule by the CONJUNCTION and CONSEQUENCE rules.

(iii) The LOOP IV rule can be derived from LOOP III rule, using the HYBRID CONJUNCTION rule and the CONSEQUENCE rule.

It suffices to note that from the first three premises

$$\vdash_{\mathrm{I}} \{p \wedge B\}\ S\ \{p\}, \quad \vdash_{\mathrm{I}} \{p \wedge B \wedge t = z\}\ S\ \{t < z\}, \quad \text{and} \quad \{p \wedge B\}\ S\ \{\mathbf{true}\}$$

of the LOOP IV rule we can derive the first two premises

$$\{p \wedge B\}\ S\ \{p\} \quad \text{and} \quad \{p \wedge B \wedge t = z\}\ S\ \{t < z\}$$

of the LOOP III rule by two applications of the HYBRID CONJUNCTION and CONSEQUENCE rules.

(*iv*) The LOOP III rule can be derived from the LOOP IV rule, using the PARTIAL CORRECTNESS rule and the CONSEQUENCE rule.

From the first two premises

$$\{p \wedge B\}\ S\ \{p\} \quad \text{and} \quad \{p \wedge B \wedge t = z\}\ S\ \{t < z\},$$

of the LOOP III rule we obtain the first two premises

$$\vdash_{\mathrm{I}} \{p \wedge B\}\ S\ \{p\} \quad \text{and} \quad \vdash_{\mathrm{I}} \{p \wedge B \wedge t = z\}\ S\ \{t < z\}$$

of the LOOP IV rule by applying the PARTIAL CORRECTNESS rule. The third premise

$$\{p \wedge B\}\ S\ \{\mathbf{true}\}$$

of the LOOP IV rule is obtained by the CONSEQUENCE rule. □

B.2 Admissibility of the Auxiliary Rules

We show the admissibility of each auxiliary rule in the proof system with the version of the LOOP rule that used this auxiliary rule according to Lemma 4.4. Additionally, we prove statement (*i*) of Lemma 4.5 because it is needed in the proof of statement (*iii*). Since the CONSEQUENCE rule is included in all these proof systems, its admissibility is trivial.

We first establish two lemmas that provide additional information about the proofs in the considered proof systems.

Lemma B.1. *Let PR be one of the proof systems I, II, III, or IV. If PR* $\vdash \varphi$*, there exists a proof of* φ *in PR with exactly one final application of the CONSEQUENCE rule.*

Proof. Since implication $\rightarrow$ is reflexive, we can always add to a given proof in *PR* one final application of the CONSEQUENCE rule. Since implication is transitive, successive applications of the CONSEQUENCE rule in a proof in *PR* can be condensed into one application. □

Lemma B.2. *Let PR be one of the proof systems I, II, III, or IV.*

(i) Suppose that $\vdash_{PR} \{p\}\ S_1;\ S_2\ \{q\}$*. Then for some assertion r*

$$\vdash_{PR} \{p\}\ S_1\ \{r\} \text{ and } \vdash_{PR} \{r\}\ S_2\ \{q\}.$$

(ii) Suppose that $\vdash_{PR} \{p\}\ \mathbf{if}\ B\ \mathbf{then}\ S_1\ \mathbf{else}\ S_2\ \mathbf{fi}\ \{q\}$*. Then*

$$\vdash_{PR} \{p \wedge B\}\ S_1\ \{q\} \text{ and } \vdash_{PR} \{p \wedge \neg B\}\ S_2\ \{q\}.$$

Proof.

(*i*) By Lemma B.1, the considered correctness formula was proved using the COMPOSITION rule followed by a single

application of the CONSEQUENCE rule. So for some assertions p_1, r, q_1, we have

$$\vdash_{PR} \{p_1\}\ S_1\ \{r\} \text{ and } \vdash_{PR} \{r\}\ S_2\ \{q_1\}$$

and the implications $p \rightarrow p_1$ and $q_1 \rightarrow q$ hold. We now get the claim by the CONSEQUENCE rule.

(ii) The argument is analogous as in (ii). □

We now turn to the proof of the Admissibility Lemma 4.5, repeating each time the statement to be proved.

Proof.

(i) The ∃-INTRODUCTION rule is admissible in the proof system II.

We proceed by induction on the structure of S and consider a proof of $\{p\}\ S\ \{q\}$ in the proof system II, where the last two steps involve an axiom or a rule of the proof system II for the top-level operator of the program S, followed by one final application of the CONSEQUENCE rule according to Lemma B.1. By the assumptions on z_0, we have $z_0 \notin var(S) \cup free(q)$.

• *Case* $S \equiv u := t$. Thus suppose $\vdash_{\mathrm{II}} \{p\}\ S\ \{q\}$. Then for some assertion q_1,

$$\vdash_{\mathrm{II}} \{q_1[u := t]\}\ S\ \{q_1\}$$

by the ASSIGNMENT axiom, and the implications $p \rightarrow q_1[u := t]$ and $q_1 \rightarrow q$ hold. Since $z_0 \in lvar$, we have $z_0 \not\equiv u$. By the assignment axiom, also

$$\{(\exists z_0 : q_1)[u := t]\}\ S\ \{\exists z_0 : q_1\}.$$

Note that the implications $\exists z_0 : p \rightarrow \exists z_0 : (q_1[u := t])$ and $\exists z_0 : q_1 \rightarrow \exists z_0 : q$ hold. Since $z_0 \not\equiv u$ and $z_0 \notin free(q)$, also the implications

$$\exists z_0 : (q_1[u := t]) \rightarrow (\exists z_0 : q_1)[u := t] \text{ and } (\exists z_0 : q) \rightarrow q$$

hold. So the CONSEQUENCE rule yields $\vdash_{\mathrm{II}} \{\exists z_0 : p\}\ S\ \{q\}$, as desired.

• *Case* $S \equiv S_1;\ S_2$. Thus suppose $\vdash_{\mathrm{II}} \{p\}\ S\ \{q\}$. By Lemma B.2$(i)$ for some assertion r,

$$\vdash_{\mathrm{II}} \{p\}\ S_1\ \{r\} \text{ and } \vdash_{\mathrm{II}} \{r\}\ S_2\ \{q\}.$$

Since $r \rightarrow \exists z_0 : r$, the CONSEQUENCE rule yields $\vdash_{\mathrm{II}} \{p\}\ S_1\ \{\exists z_0 : r\}$. Since $z_0 \notin free(\exists z_0 : r)$, the induction hypothesis yields $\vdash_{\mathrm{II}} \{\exists z_0 : p\}\ S_1\ \{\exists z_0 : r\}$. Since $z_0 \notin free(q)$, by the induction hypothesis, also $\vdash_{\mathrm{II}} \{\exists z_0 : r\}\ S_2\ \{q\}$. Thus by the COMPOSITION rule, $\vdash_{\mathrm{II}} \{\exists z_0 : p\}\ S\ \{q\}$, as desired.

• *Case* $S \equiv$ **if** B **then** S_1 **else** S_2 **fi**. Thus suppose $\vdash_{\mathrm{II}} \{p\}\ S\ \{q\}$. Then by Lemma B.2$(ii)$,

$$\vdash_{\mathrm{II}} \{p \wedge B\}\ S_1\ \{q\} \text{ and } \vdash_{\mathrm{II}} \{p \wedge \neg B\}\ S_2\ \{q\}.$$

By the induction hypothesis,

$$\vdash_{\mathrm{II}} \{\exists z_0 : (p \wedge B)\}\ S_1\ \{q\} \text{ and } \vdash_{\mathrm{II}} \{\exists z_0 : (p \wedge \neg B)\}\ S_2\ \{q\}.$$

Since $z_0 \notin var(B)$, the CONSEQUENCE rule yields

$$\vdash_{\mathrm{II}} \{(\exists z_0 : p) \wedge B\}\ S_1\ \{q\} \text{ and } \vdash_{\mathrm{II}} \{(\exists z_0 : p) \wedge \neg B\}\ S_2\ \{q\}.$$

By the CONDITIONAL rule, $\vdash_{\mathrm{II}} \{\exists z_0 : p\}\ S\ \{q\}$, as desired.

• *Case* $S \equiv$ **while** B **do** S_0 **od**. Thus suppose $\vdash_{\mathrm{II}} \{p\}\ S\ \{q\}$. By the assumption about t, we have $z_0 \notin var(t)$ and without loss of generality we can assume $z_0 \neq z$. Then for some assertion p_0 and an appropriate bound function t and variable z,

$$\vdash_{\mathrm{II}} \{p_0 \wedge B \wedge t = z\}\ S_0\ \{p_0 \wedge t < z\},$$

and the implications $p \rightarrow p_0$, $p_0 \rightarrow t \geq 0$, $(p_0 \wedge \neg B) \rightarrow q$ hold.

Since $(p_0 \wedge t < z) \rightarrow ((\exists z_0 : p_0) \wedge t < z)$, the CONSEQUENCE rule yields

$$\vdash_{\mathrm{II}} \{p_0 \wedge B \wedge t = z\}\ S_0\ \{(\exists z_0 : p_0) \wedge t < z\}.$$

By the induction hypothesis, the assumption about z_0, and the CONSEQUENCE rule

$$\vdash_{\mathrm{II}} \{(\exists z_0 : p_0) \wedge B \wedge t = z\}\ S_0\ \{(\exists z_0 : p_0) \wedge t < z\}.$$

So the LOOP II rule yields

$$\vdash_{\mathrm{II}} \{\exists z_0 : p_0\}\ S\ \{(\exists z_0 : p_0) \wedge \neg B\}.$$

Further, since $(p_0 \wedge \neg B) \rightarrow q$ holds and $z_0 \notin (free(q) \cup var(B))$, also the implication $((\exists z_0 : p_0) \wedge \neg B) \rightarrow q$ holds. So a final application of the CONSEQUENCE rule yields $\vdash_{\mathrm{II}} \{\exists z_0 : p\}\ S\ \{q\}$, as desired.

(ii) The $\exists$-INTRODUCTION rule is admissible in the proof system III.

Again, we proceed by induction on the structure of S, but now consider a proof of $\{p\}\ S\ \{q\}$ in the proof system III, where the last two steps involve the axiom or rule of the proof system III for the top-level operator of the program S, followed by one final application of the CONSEQUENCE rule according to Lemma B.1.

Except for the **while** statement, all cases are analogous, with $\vdash_{\mathrm{II}}$ replaced by $\vdash_{\mathrm{III}}$. The case of the **while** statement differs from (i) only in that one now considers two correctness formulas in the premise of the LOOP III rule instead of one. As above, let $S \equiv$ **while** B **do** S_0 **od** and suppose $\vdash_{\mathrm{III}} \{p\}\ S\ \{q\}$. By the assumption about t, we have $z_0 \notin var(t)$ and without loss of generality we can assume $z_0 \neq z$. Then for some assertion p_0 and an appropriate bound function t and variable z,

$$\vdash_{\mathrm{III}} \{p_0 \wedge B\}\ S_0\ \{p_0\},$$

$$\vdash_{\mathrm{III}} \{p_0 \wedge B \wedge t = z\}\ S_0\ \{t < z\},$$

and the implications $p \rightarrow p_0$, $p_0 \rightarrow t \geq 0$, $(p_0 \wedge \neg B) \rightarrow q$ hold.

Since $p_0 \rightarrow \exists z_0 : p_0$, the CONSEQUENCE rule yields

$$\vdash_{\mathrm{III}} \{p_0 \wedge B\}\ S_0\ \{\exists z_0 : p_0\}.$$

By the induction hypothesis, the assumption about z_0, and the CONSEQUENCE rule both

$$\vdash_{\mathrm{III}} \{(\exists z_0 : p_0) \wedge B\}\ S_0\ \{\exists z_0 : p_0\}$$

and

$$\vdash_{\mathrm{III}} \{(\exists z_0 : p_0) \wedge B \wedge t = z\}\ S_0\ \{t < z\}.$$

So the LOOP III rule yields

$$\vdash_{\mathrm{III}} \{\exists z_0 : p_0\}\ S\ \{(\exists z_0 : p_0) \wedge \neg B\}.$$

Further, since $(p_0 \wedge \neg B) \rightarrow q$ holds and $z_0 \notin (free(q) \cup var(B))$, also the implication $((\exists z_0 : p_0) \wedge \neg B) \rightarrow q$ holds. So a final application of the CONSEQUENCE rule yields $\vdash_{\mathrm{III}} \{\exists z_0 : p\}\ S\ \{q\}$, as desired.

(iii) The CONJUNCTION rule is admissible in the proof system II.

We proceed by induction on the structure of S and for $i = 1, 2$ consider proofs of $\{p_i\}\ S\ \{q_i\}$ in the proof system II, where the last two steps involve the axiom or rule of the proof system II for the top-level operator of the program S, followed by one final application of the CONSEQUENCE rule according to Lemma B.1.

• *Case* $S \equiv u := t$. Thus suppose $\vdash_{\mathrm{II}} \{p_1\}\ S\ \{q_1\}$ and $\vdash_{\mathrm{II}} \{p_2\}\ S\ \{q_2\}$. Then for some assertions q_{01} and q_{02}, by the ASSIGNMENT axiom, both

$$\vdash_{\mathrm{II}} \{q_{01}[u := t]\}\ S\ \{q_{01}\} \ \text{ and } \ \vdash_{\mathrm{II}} \{q_{02}[u := t]\}\ S\ \{q_{02}\},$$

and the implications $p_1 \rightarrow q_{01}[u := t]$, $q_{01} \rightarrow q_1$ and $p_2 \rightarrow q_{02}[u := t]$, $q_{02} \rightarrow q_2$ hold. The ASSIGNMENT axiom also yields

$$\{q_{01}[u := t] \wedge q_{02}[u := t]\}\ S\ \{q_{01} \wedge q_{02}\}.$$

By the implications $(p_1 \wedge p_2) \rightarrow (q_{01}[u := t] \wedge q_{02}[u := t])$ and $(q_{01} \wedge q_{02}) \rightarrow (q_1 \wedge q_2)$, the CONSEQUENCE rule yields

$$\vdash_{\mathrm{II}} \{p_1 \wedge p_2\}\ S\ \{q_1 \wedge q_2\},$$

as desired.

• *Case* $S \equiv S_1;\ S_2$. Thus suppose $\vdash_{\mathrm{II}} \{p_1\}\ S\ \{q_1\}$ and $\vdash_{\mathrm{II}} \{p_2\}\ S\ \{q_2\}$. Then by Lemma B.2$(i)$ for some assertions r_1 and r_2

$$\vdash_{\mathrm{II}} \{p_1\}\ S_1\ \{r_1\} \ \text{ and } \ \vdash_{\mathrm{II}} \{r_1\}\ S_2\ \{q_1\},$$
$$\vdash_{\mathrm{II}} \{p_2\}\ S_1\ \{r_2\} \ \text{ and } \ \vdash_{\mathrm{II}} \{r_2\}\ S_2\ \{q_2\}.$$

By the induction hypothesis,

$$\vdash_{\mathrm{II}} \{p_1 \wedge p_2\}\ S_1\ \{r_1 \wedge r_2\} \ \text{ and } \ \vdash_{\mathrm{II}} \{r_1 \wedge r_2\}\ S_1\ \{q_1 \wedge q_2\},$$

so by the COMPOSITION rule,

$$\vdash_{\mathrm{II}} \{p_1 \wedge p_2\}\ S_1;\ S_2\ \{q_1 \wedge q_2\},$$

as desired.

• *Case* $S \equiv$ **if** B **then** S_1 **else** S_2 **fi**. Thus suppose $\vdash_{\mathrm{II}} \{p_1\}\ S\ \{q_1\}$ and $\vdash_{\mathrm{II}} \{p_2\}\ S\ \{q_2\}$. Then by Lemma B.2$(ii)$,

$$\vdash_{\mathrm{II}} \{p_1 \wedge B\}\ S_1\ \{q_1\} \ \text{ and } \ \vdash_{\mathrm{II}} \{p_1 \wedge \neg B\}\ S_2\ \{q_1\},$$

$$\vdash_{\mathrm{II}} \{p_2 \wedge B\}\ S_1\ \{q_2\} \ \text{ and } \ \vdash_{\mathrm{II}} \{p_2 \wedge \neg B\}\ S_2\ \{q_2\}.$$

By the induction hypothesis,

$$\vdash_{\mathrm{II}} \{p_1 \wedge p_2 \wedge B\}\ S_1\ \{q_1 \wedge q_2\}$$

and

$$\vdash_{\mathrm{II}} \{p_1 \wedge p_2 \wedge \neg B\}\ S_2\ \{q_1 \wedge q_2\}.$$

So by the CONDITIONAL rule,

$$\vdash_{\mathrm{II}} \{p_1 \wedge p_2\}\ \textbf{if}\ B\ \textbf{then}\ S_1\ \textbf{else}\ S_2\ \textbf{fi}\ \{q_1 \wedge q_2\},$$

as desired.

• *Case* $S \equiv$ **while** B **do** S_0 **od**. Suppose $\vdash_{\mathrm{II}} \{p_1\}\ S\ \{q_1\}$ and $\vdash_{\mathrm{II}} \{p_2\}\ S\ \{q_2\}$. Then for some assertions p_{01}, p_{02}, appropriate bound functions t_1, t_2 and variables z_1, z_2,

$$\vdash_{\mathrm{II}} \{p_{01} \wedge B \wedge t_1 = z_1\}\ S_0\ \{p_{01} \wedge t_1 < z_1\},$$

$$\vdash_{\mathrm{II}} \{p_{02} \wedge B \wedge t_2 = z_2\}\ S_0\ \{p_{02} \wedge t_2 < z_2\},$$

the implications $p_{01} \rightarrow t_1 \geq 0$, $p_1 \rightarrow p_{01}$, $(p_{01} \wedge \neg B) \rightarrow q_1$ and $p_{02} \rightarrow t_2 \geq 0$, $p_2 \rightarrow p_{02}$, $(p_{02} \wedge \neg B) \rightarrow q_2$ hold.

Without loss of generality we can assume that $z_2 \notin \{z_1\} \cup \mathit{free}(p_{01})$. By the induction hypothesis,

$$\vdash_{\mathrm{II}} \{p_{01} \wedge p_{02} \wedge B \wedge t_1 = z_1 \wedge t_2 = z_2\}\ S_0\ \{p_{01} \wedge p_{02} \wedge t_1 < z_1 \wedge t_2 < z_2\}.$$

It suffices to consider one bound function, say t_1. Formally, we show this as follows. By the CONSEQUENCE rule,

$$\vdash_{\mathrm{II}} \{p_{01} \wedge p_{02} \wedge B \wedge t_1 = z_1 \wedge t_2 = z_2\}\ S_0\ \{p_{01} \wedge p_{02} \wedge t_1 < z_1\}.$$

Now, an application of the $\exists$-INTRODUCTION rule, which is admissible in the proof system II according to part (i) of this lemma, followed by an application of the CONSEQUENCE rule yields

$$\vdash_{\mathrm{II}} \{p_{01} \wedge p_{02} \wedge B \wedge t_1 = z_1 \wedge \exists z_2 : t_2 = z_2\}\ S_0\ \{p_{01} \wedge p_{02} \wedge t_1 < z_1\}.$$

A further application of the CONSEQUENCE rule yields

$$\vdash_{\mathrm{II}} \{p_{01} \wedge p_{02} \wedge B \wedge t_1 = z_1\}\ S_0\ \{p_{01} \wedge p_{02} \wedge t_1 < z_1\}.$$

Then by the LOOP II rule,

$$\vdash_{\mathrm{II}} \{p_{01} \wedge p_{02}\}\ \textbf{while}\ B\ \textbf{do}\ S_0\ \textbf{od}\ \{p_{01} \wedge p_{02} \wedge \neg B\}.$$

The implications above yield $(p_1 \wedge p_2) \rightarrow (p_{01} \wedge p_{02})$ and $(p_{01} \wedge p_{02} \wedge \neg B) \rightarrow (q_1 \wedge q_2)$. Thus by the CONSEQUENCE rule,

$$\vdash_{\mathrm{II}} \{p_1 \wedge p_2\}\ \textbf{while}\ B\ \textbf{do}\ S_0\ \textbf{od}\ \{q_1 \wedge q_2\},$$

as desired.

(iv) The HYBRID CONJUNCTION rule is admissible in the proof system III, that is, if $\vdash_{\mathrm{I}} \{p_1\}\ S\ \{q_1\}$ and $\vdash_{\mathrm{III}} \{p_2\}\ S\ \{q_2\}$, then $\vdash_{\mathrm{III}} \{p_1 \wedge p_2\}\ S\ \{q_1 \wedge q_2\}$.

The proof is analogous to the proof of (iii). The only case that is somewhat different is the one concerned with the **while** statement. So we only deal with

• *Case* $S \equiv$ **while** B **do** S_0 **od**. Suppose $\vdash_{\mathrm{I}} \{p_1\}\ S\ \{q_1\}$ and $\vdash_{\mathrm{III}} \{p_2\}\ S\ \{q_2\}$. Then for some assertions p_{01}, p_{02} and an appropriate bound function t and variable z,

$$\vdash_{\mathrm{I}} \{p_{01} \wedge B\}\ S_0\ \{p_{01}\},$$

$$\vdash_{\mathrm{III}} \{p_{02} \wedge B\}\ S_0\ \{p_{02}\},$$

$$\vdash_{\mathrm{III}} \{p_{02} \wedge B \wedge t = z\}\ S_0\ \{t < z\},$$

and the implications

$$p_1 \rightarrow p_{01},\ (p_{01} \wedge \neg B) \rightarrow q_1,\ p_{02} \rightarrow t \geq 0,\ p_2 \rightarrow p_{02},\ (p_{02} \wedge \neg B) \rightarrow q_2$$

hold. By the induction hypothesis,

$$\vdash_{\mathrm{III}} \{p_{01} \wedge p_{02} \wedge B\}\ S_0\ \{p_{01} \wedge p_{02}\},$$

and by the induction hypothesis combined with the CONSEQUENCE rule,

$$\vdash_{\mathrm{III}} \{p_{01} \wedge p_{02} \wedge B \wedge t = z\}\ S_0\ \{t < z\}.$$

Since $p_{01} \wedge p_{02} \rightarrow t \geq 0$, the LOOP III rule yields

$$\vdash_{\mathrm{III}} \{p_{01} \wedge p_{02}\}\ \textbf{while}\ B\ \textbf{do}\ S_0\ \textbf{od}\ \{p_{01} \wedge p_{02} \wedge \neg B\}.$$

The implications above yield $(p_1 \wedge p_2) \rightarrow (p_{01} \wedge p_{02})$ and $(p_{01} \wedge p_{02} \wedge \neg B) \rightarrow (q_1 \wedge q_2)$. Thus by the CONSEQUENCE rule,

$$\vdash_{\mathrm{III}} \{p_1 \wedge p_2\}\ \textbf{while}\ B\ \textbf{do}\ S_0\ \textbf{od}\ \{q_1 \wedge q_2\},$$

as desired.

(v) The PARTIAL CORRECTNESS rule is admissible in the proof system IV.

Suppose $\vdash_{\mathrm{IV}} \{p\}\ S\ \{q\}$. Consider a corresponding proof of $\{p\}\ S\ \{q\}$ in the proof system IV. Each application of the LOOP IV rule requires the establishment of four premises, of which the first one is exactly the premise an application of the LOOP I rule depends on. If we eliminate in the given proof of $\{p\}\ S\ \{q\}$ in system IV all parts establishing the last three premises in each application of the LOOP IV rule, we obtain a proof of $\{p\}\ S\ \{q\}$ in the proof system I using only the LOOP I rule. Thus $\vdash_{\mathrm{I}} \{p\}\ S\ \{q\}$. □

B.3 Instantiating Lemma 4.3

Finally, we explain how to obtain the Equivalence Theorem 2.1 by instantiating Lemma 4.3 using the Derivability Lemma 4.4 and the Admissibility Lemma 4.5. We show one illustrative case: proof system II ≈ proof system III.

Let T be the proof system from Appendix A without the LOOP I rule. Then, using Lemma 4.4(i) and Lemma 4.5(ii), instantiating Lemma 4.3 yields

$$T \cup \text{LOOP II} \ \preceq\ T \cup \text{LOOP III}.$$

Also, using Lemma 4.4(ii) and Lemma 4.5(iii), instantiating Lemma 4.3 yields

$$T \cup \text{LOOP III} \ \preceq T \cup \text{LOOP II}.$$

So altogether $T \cup \text{LOOP II} \ \approx\ T \cup \text{LOOP III}$, which means proof system II ≈ proof system III.

C Admissibility of the LOOP B versions

We establish here the following result that clarifies the proof-theoretic status of the B versions of the LOOP rules. Let us denote the proof systems using these versions with a B in the end.

Theorem C.1. *Each proof system αB is equivalent to the proof system α, where $\alpha \in \{\mathrm{II}, \mathrm{III}, \mathrm{IV}\}$.*

Corollary C.2. *Each* LOOP *αB rule is admissible in the proof system α, where $\alpha \in \{\mathrm{II}, \mathrm{III}, \mathrm{IV}\}$.*

We prove Theorem C.1 in detail for the case $\alpha = \mathrm{III}$. First we show:

Lemma C.3. *The* LOOP III *B rule can be derived from the* LOOP III *using the* HYBRID CONJUNCTION *rule, the* ∃-INTRODUCTION *rule, and the* CONSEQUENCE *rule.*

Proof. Suppose that for some loop invariant p and bound function t

$$\{p \wedge B\}\ S\ \{p\}, \tag{2}$$

$$\{p \wedge B \wedge t = z\}\ S\ \{t < z\} \tag{3}$$

and $p \wedge B \rightarrow t \geq 0$ hold. By the INVARIANCE axiom in the proof system I (see Note 1), we have

$$\vdash_{\mathrm{I}} \{z' = z + 1 \wedge 0 < z'\}\ S\ \{z' = z + 1 \wedge 0 < z'\},$$

where z' is a fresh logical variable. Applying the HYBRID CONJUNCTION rule to this correctness formula and (3), we obtain

$$\{z' = z + 1 \wedge 0 < z' \wedge p \wedge B \wedge t = z\}\ S\ \{z' = z + 1 \wedge 0 < z' \wedge t < z\}. \tag{4}$$

Next, note that for the bound function $t' \equiv$ **if** B **then** $t + 1$ **else** 0 **fi** the following implications hold:

$$z' = z + 1 \wedge 0 < z' \wedge p \wedge B \wedge t' = z' \rightarrow z' = z + 1 \wedge 0 < z' \wedge p \wedge B \wedge t = z$$

and

$$z' = z + 1 \wedge 0 < z' \wedge t < z \rightarrow t' < z',$$

so applying the CONSEQUENCE rule to (4) yields

$$\{z' = z + 1 \wedge 0 < z' \wedge p \wedge B \wedge t' = z'\}\ S\ \{t' < z'\}.$$

Applying next the ∃-INTRODUCTION rule, we get

$$\{\exists z : (z' = z + 1 \wedge 0 < z' \wedge p \wedge B \wedge t' = z')\}\ S\ \{t' < z'\}. \tag{5}$$

Note now the following sequence of implications:

$$\begin{aligned}
&p \wedge B \wedge t' = z' \rightarrow \\
&0 \leq t \wedge p \wedge B \wedge t' = z' \rightarrow \\
&0 \leq t \wedge t' = t + 1 \wedge p \wedge B \wedge t' = z' \rightarrow \\
&0 < z' \wedge p \wedge B \wedge t' = z' \rightarrow \\
&(\exists z : z' = z + 1) \wedge 0 < z' \wedge p \wedge B \wedge t' = z' \rightarrow \\
&\exists z : (z' = z + 1 \wedge 0 < z' \wedge p \wedge B \wedge t' = z').
\end{aligned}$$

An application of the CONSEQUENCE rule to (5) then yields

$$\{p \wedge B \wedge t' = z'\}\ S\ \{t' < z'\}, \tag{6}$$

Further, we have $p \wedge B \rightarrow t \geq 0 \wedge t' = t + 1$ and $p \wedge \neg B \rightarrow t' = 0$, so $p \rightarrow t' \geq 0$ holds. This allows us to apply the LOOP III rule to (2), (6), and the last implication to establish

$$\{p\}\ \textbf{while}\ B\ \textbf{do}\ S\ \textbf{od}\ \{p \wedge \neg B\}.$$

□

By Lemma 4.5, the auxiliary rules needed in Lemma C.3 are admissible in proof system III. The converse of Lemma C.3 is immediate.

Lemma C.4. *The* LOOP III *rule can be derived from the* LOOP III B.

Proof. Suppose that for some loop invariant p and bound function t

$$\{p \wedge B\}\ S\ \{p\},\ \{p \wedge B \wedge t = z\}\ S\ \{t < z\},\ \text{and}\ p \rightarrow t \geq 0.$$

Since the last condition implies $p \wedge B \rightarrow t \geq 0$, an application of the LOOP III B rule yields

$$\{p\}\ \textbf{while}\ B\ \textbf{do}\ S\ \textbf{od}\ \{p \wedge \neg B\}.$$

□

Finally, by instantiating Lemma 4.3 with the Lemmas C.3, C.4, and 4.5, we obtain that the proof system III B is equivalent to the proof system III. For the cases II and IV, the equivalence proofs require lemmas corresponding to Lemma C.3 and Lemma C.4, which can be proven analogously, though it has to be shown along the lines of Lemma 4.5 that the HYBRID CONJUNCTION rule is admissible in the proof systems II and IV, and that the ∃-INTRODUCTION rule is admissible in the proof system IV.

Received Day Month 2026